

TinyDB Query Engine and SQL Parser

A complete pipeline from raw SQL text to an executed result:

```
"SELECT name FROM employee WHERE salary > 50000"
  |
  v
SqlLexer      -- text -> tokens
  |
  v
SqlParser     -- tokens -> SqlStatement (an immutable AST)
  |
  v
QueryEngine   -- Level 1: execute the AST directly
               Level 2: hand it to a Planner instead
  |
  v
StorageEngine -- built in the companion Storage Engine guide
```

1. Implement Lexer

- 1.1. Token Types [02]
- 1.2. Initial Grammar [02]
- 1.3. SqlLexer [03]
- 1.4. Domain Models [05]
 - Data Type, Column, Table Schema
 - Row, Catalog

2. AST

- 2.1. SqlStatement [06]
- 2.2. SelectStatement [06]
- 2.3. InsertStatement [06]
- 2.4. UpdateStatement [06]
- 2.5. DeleteStatement [06]
- 2.6. CreateTableStatement [06]
- 2.7. DropTableStatement [06]

3. Parser

- 3.1. SqlParser [07]
- 3.2. Expression AST [10]
- 3.3. Expression Evaluator [11]
- 3.4. QueryResult and TransactionContext [12]

4. StorageEngine

- 4.1. SelectExecutor [14]
- 4.2. InsertExecutor [15]
- 4.3. UpdateExecutor [16]
- 4.4. DeleteExecutor [16]
- 4.5. CreateTableExecutor [17]
- 4.6. DropTableExecutor [17]
- 4.6. DropTableExecutor [17]
- 4.7. QueryDispatcher: [18]
- 4.8. ANTLR Alternative [19]

5. Planner

- 5.1. Plan Interface [21]
- 5.2. TablePlan [21]
- 5.3. SelectPlan [21]
- 5.4. ProjectPlan [22]
- 5.5. BasicQueryPlanner [22]
- 5.6. UpdatePlanner [23]
- 5.7. BasicPlanner Facade [24]

6. QueryEngine

- 6.1. BasicQueryEngine [25]
- 6.2. Revisiting the [26]

7. The Scan Interface: Streaming

- Instead of Materializing [26]
 - TableScan [27]
 - SelectScan [28]
 - ProjectScan [28]
 - Extending the Expression Evaluator to Work Against a Scan

8. Why Streaming Beats Materializing

9. The Catalog Interface [30]

10. TablePhysicalLayout [30]

11. PersistentCatalog [31]

Project Structure

```
tinydb-query-engine/
├── src/main/java/com/tinydb/sql/
│   ├── lexer/
│   │   ├── Token.java, TokenType.java
│   │   └── SqlLexer.java
│   ├── ast/
│   │   ├── SqlStatement.java // sealed interface
│   │   ├── SelectStatement.java, InsertStatement.java, UpdateStatement.java,
│   │   ├── DeleteStatement.java, CreateTableStatement.java, DropTableStatement.java
│   │   ├── Expression, Comparison.java, And.java, Or.java
│   │   └── ExpressionEvaluator.java
│   ├── parser/
│   │   └── SqlParser.java
│   ├── executor/
│   │   ├── StatementExecutor.java
│   │   ├── SelectExecutor, InsertExecutor, UpdateExecutor,
│   │   ├── DeleteExecutor, CreateTableExecutor, DropTableExecutor
│   │   └── QueryEngine.java
│   ├── plan/
│   │   ├── Plan.java, TablePlan.java, SelectPlan.java, ProjectPlan.java
│   │   ├── Scan.java, TableScan.java, SelectScan.java, ProjectScan.java
│   │   └── QueryPlanner, BasicQueryPlanner, UpdatePlanner, BasicUpdatePlanner
│   ├── engine/
│   │   ├── BasicPlanner.java
│   │   └── BasicQueryEngine.java
│   └── catalog/
│       ├── Catalog.java, TableSchema.java, Column.java, DataType.java
│       ├── TablePhysicalLayout.java
│       └── PersistentCatalog.java
└── src/test/java/com/tinydb/sql/
    ├── SqlLexerTest.java
    ├── SqlParserTest.java
    ├── ExpressionEvaluatorTest.java
    ├── QueryEngineTest.java
    └── BasicQueryPlannerTest.java
```

High-Level Architecture

Lexer → Parser → AST → Executors/Planner → Scan Pipeline

Two execution strategies are built in this guide, deliberately in sequence, not as alternatives to choose between arbitrarily:

Level 1 (Part 3): SqlStatement -> Executor -> StorageEngine directly

Simple, correct, materializes full result sets in memory.

Level 2 (Part 4-5): SqlStatement -> Planner -> Plan -> Scan -> StorageEngine

More moving parts, streams rows one at a time, and is the shape a Cost-Based Optimizer (Part 6) can eventually plug into.

Phase 1 — Tokens and the SQL Lexer

A lexer turns raw text into a flat sequence of tokens — the smallest meaningful pieces of the language (keywords, identifiers, literals, operators, punctuation) — with no notion yet of how they combine into a statement. That structural question is entirely the parser's job (Part 2).

1.1. TokenType

```
public enum TokenType {  
    // Keywords  
    SELECT, FROM, WHERE, INSERT, INTO, VALUES, UPDATE, SET, DELETE,  
    CREATE, TABLE, DROP, AND, OR, PRIMARY, KEY,  
    // Literals  
    IDENTIFIER, NUMBER, STRING,  
    // Punctuation and operators  
    STAR, COMMA, LPAREN, RPAREN, EQ, NEQ, GT, GTE, LT, LTE,  
    // End of input  
    EOF  
}
```

```
public record Token(TokenType type, String text) { }
```

1.2. The SQL Grammar We Support (Start Small)

```
statement  
    := select | insert | update | delete | createTable | dropTable  
  
select  
    := SELECT selectList FROM identifier whereClause?  
  
selectList  
    := STAR  
    | identifier (COMMA identifier)*  
  
whereClause  
    := WHERE expression  
  
expression  
    := comparison  
    | expression AND expression  
    | expression OR expression  
  
comparison  
    := identifier operator value  
  
operator  
    := '=' | '!=' | '>' | '>=' | '<' | '<='
```

1.3. Lexer

```
public final class SqlLexer {
    private final String sql;
    private int position = 0;

    public SqlLexer(String sql) { this.sql = sql; }

    public List<Token> tokenize() {
        List<Token> tokens = new ArrayList<>();
        Token token;
        while ((token = nextToken()).type() != TokenType.EOF) {
            tokens.add(token);
        }
        tokens.add(token);
        return tokens;
    }

    private Token nextToken() {
        skipWhitespace();
        if (position >= sql.length()) return new Token(TokenType.EOF, "");
        char c = sql.charAt(position);
        if (Character.isLetter(c) || c == '_') return readIdentifierOrKeyword();
        if (Character.isDigit(c)) return readNumber();
        if (c == "\"") return readStringLiteral();

        return readOperatorOrPunctuation();
    }

    private void skipWhitespace() {
        while (position < sql.length() && Character.isWhitespace(sql.charAt(position))) position++;
    }

    private Token readIdentifierOrKeyword() {
        int start = position;
        while (position < sql.length() && (Character.isLetterOrDigit(sql.charAt(position)) || sql.charAt(position) == '_')) {
            position++;
        }
        String text = sql.substring(start, position);
        TokenType keyword = keywordOrNull(text.toUpperCase());
        return new Token(keyword != null ? keyword : TokenType.IDENTIFIER, text);
    }

    private Token readNumber() {
        int start = position;
        while (position < sql.length() && Character.isDigit(sql.charAt(position))) position++;
        return new Token(TokenType.NUMBER, sql.substring(start, position));
    }

    private Token readStringLiteral() {
        position++; // consume the opening quote
        int start = position;
        while (position < sql.length() && sql.charAt(position) != "\"") position++;
        String text = sql.substring(start, position);
        position++; // consume the closing quote
        return new Token(TokenType.STRING, text);
    }

    private TokenType keywordOrNull(String upper) {
        return switch (upper) {
            case "SELECT" -> TokenType.SELECT;
            case "FROM" -> TokenType.FROM;
            case "WHERE" -> TokenType.WHERE;
            case "INSERT" -> TokenType.INSERT;
            case "INTO" -> TokenType.INTO;
        };
    }
}
```

```

        case "VALUES" -> TokenType.VALUES;
        case "UPDATE" -> TokenType.UPDATE;
        case "SET" -> TokenType.SET;
        case "DELETE" -> TokenType.DELETE;
        case "CREATE" -> TokenType.CREATE;
        case "TABLE" -> TokenType.TABLE;
        case "DROP" -> TokenType.DROP;
        case "AND" -> TokenType.AND;
        case "OR" -> TokenType.OR;
        case "PRIMARY" -> TokenType.PRIMARY;
        case "KEY" -> TokenType.KEY;
        default -> null; // not a keyword -- caller falls back to IDENTIFIER
    };

    private Token readOperatorOrPunctuation() {
        char c = sql.charAt(position);
        char next = position + 1 < sql.length() ? sql.charAt(position + 1) : '\0';

        return switch (c) {
            case '*' -> single(TokenType.STAR);
            case ',' -> single(TokenType.COMMA);
            case '(' -> single(TokenType.LPAREN);
            case ')' -> single(TokenType.RPAREN);
            case '=' -> single(TokenType.EQ);
            case '!' when next == '=' -> doubleChar(TokenType.NEQ);
            case '>' when next == '=' -> doubleChar(TokenType.GTE);
            case '>' -> single(TokenType.GT);
            case '<' when next == '=' -> doubleChar(TokenType.LTE);
            case '<' -> single(TokenType.LT);
            default -> throw new IllegalArgumentException("Unexpected character: " + c);
        };
    }

    private Token single(TokenType type) {
        String text = sql.substring(position, position + 1);
        position++;
        return new Token(type, text);
    }

    private Token doubleChar(TokenType type) {
        String text = sql.substring(position, position + 2);
        position += 2;
        return new Token(type, text);
    }
}

```

1.4. The Domain Model: Row, TableSchema, Column, and DataType

Before the AST can reference a table's shape (§12's CREATE TABLE parsing needs Column/DataType/TableSchema immediately), the small set of plain data types every later section builds on needs to exist:

```
// catalog/DataType.java
```

```
public enum DataType {  
    INT, BIGINT, BOOLEAN, VARCHAR, DECIMAL, DATE, TIMESTAMP  
}
```

```
// catalog/Column.java
```

```
public record Column(String name, DataType type, boolean nullable) { }
```

```
// catalog/TableSchema.java
```

```
public record TableSchema(String tableName, List<Column> columns) {  
    public List<String> fieldNames() {  
        return columns.stream().map(Column::name).toList();  
    }  
  
    public TableSchema project(List<String> fieldNames) {  
        List<Column> projected = columns.stream().filter(c -> fieldNames.contains(c.name())).toList();  
        return new TableSchema(tableName, projected);  
    }  
}
```

```
// A row is deliberately a thin wrapper over a Map for this guide's purposes  
// companion Storage Engine guide for the REAL binary,  
// on-disk row representation this  
// in-memory shape stands in for at execution time.
```

```
public final class Row {  
    private final Map<String, Object> values;  
  
    public Row(Map<String, Object> values) { this.values = Map.copyOf(values); }  
  
    public Object get(String column) { return values.get(column); }  
    public Set<String> columnNames() { return values.keySet(); }  
}
```

```
// catalog/Catalog.java
```

```
public interface Catalog {  
    void createTable(TableSchema schema);  
    void dropTable(String tableName);  
    TableSchema getTable(String tableName);  
    boolean tableExists(String tableName);  
    List<TableSchema> listTables();  
}
```

Phase 2 — The SqlStatement AST

The parser's entire output is one value: a `SqlStatement`. Making it a sealed interface is the single most consequential design decision in this guide — every later dispatch (executors in Part 3, the planner in Part 4) gets compiler-enforced exhaustiveness from this one choice:

```
// ast/SqlStatement.java
public sealed interface SqlStatement
    permits SelectStatement, InsertStatement, UpdateStatement,
           DeleteStatement, CreateTableStatement, DropTableStatement {
}
```

```
record SelectStatement(
    List<String> columns,
    String table,
    Expression where // null if there was no WHERE clause
) implements SqlStatement { }
```

```
record InsertStatement(
    String table,
    List<String> columns,
    List<Object> values
) implements SqlStatement { }
```

```
public record UpdateStatement(
    String table,
    Map<String, Object> assignments,
    Expression where
) implements SqlStatement { }
```

```
public record CreateTableStatement(
    TableSchema schema
) implements SqlStatement { }
```

```
public record DeleteStatement(
    String table,
    Expression where
) implements SqlStatement { }
```

```
public record DropTableStatement(
    String table
) implements SqlStatement { }
```

Phase 3 — Recursive-Descent Parsing

From Tokens to SqlStatement

3.1. SqlParser

```
// parser/SqlParser.java
public final class SqlParser {
    private final List<Token> tokens;
    private int position = 0;

    public SqlParser(List<Token> tokens) {
        this.tokens = tokens;
    }

    public SqlStatement parseStatement() {
        return switch (peek().type()) {
            case SELECT -> parseSelect();
            case INSERT -> parseInsert();
            case UPDATE -> parseUpdate();
            case DELETE -> parseDelete();
            case CREATE -> parseCreateTable();
            case DROP -> parseDropTable();
            default -> throw new IllegalArgumentException("Unexpected token: " + peek());
        };
    }

    // ---- Token stream helpers ----

    private Token peek() { return tokens.get(position); }

    private Token advance() { return tokens.get(position++); }

    private Token expect(TokenType type) {
        Token token = advance();
        if (token.type() != type) {
            throw new IllegalArgumentException("Expected " + type + " but found " + token);
        }
        return token;
    }

    private boolean check(TokenType type) { return peek().type() == type; }
}
```

peek/advance/expect are the entire vocabulary a recursive-descent parser needs — look at the next token without consuming it, consume the next token, or consume it while asserting it's the type you expected. Every grammar rule below is built from exactly these three operations.

// SqlParser.java, continued.....

Import static TokenType.IDENTIFIER, WHERE;

```
private SelectStatement parseSelect() {
    expect(TokenType.SELECT);
    List<String> columns = parseSelectList();
    expect(TokenType.FROM);
    String table = expect(IDENTIFIER).text();
    Expression where = check(WHERE) ?
    parseWhereClause() : null;
    return new SelectStatement(columns, table,
    where);
}
```

```
private List<String> parseSelectList() {
    if (check(TokenType.STAR)) {
        advance();
        return List.of("");
    }
    List<String> columns = new ArrayList<>();
    columns.add(expect(IDENTIFIER).text());
    while (check(TokenType.COMMA)) {
        advance();
        columns.add(expect(IDENTIFIER).text());
    }
    return columns;
}
```

```
private List<String> parseIdentifierList() {
    List<String> identifiers = new ArrayList<>();
    identifiers.add(expect(IDENTIFIER).text());
    while (check(TokenType.COMMA)) {
        advance();
        identifiers.add(expect(IDENTIFIER).text());
    }
    return identifiers;
}
```

```
private Object parseLiteral() {
    Token token = advance();
    return switch (token.type()) {
        case NUMBER -> parseLong(token.text());
        case STRING -> token.text();
        default -> throw new
    IllegalArgumentException("Expected a literal but
    found " + token);
    };
}
```

```
private Map<String, Object> parseAssignment()
{
    String column =
    expect(TokenType.IDENTIFIER).text();
    expect(TokenType.EQ);
    Object value = parseLiteral();
    return Map.of(column, value);
}
```

```
private InsertStatement parseInsert() {
    expect(TokenType.INSERT);
    expect(TokenType.INTO);
    String table = expect(IDENTIFIER).text();

    expect(TokenType.LPAREN);
    List<String> columns = parseIdentifierList();
    expect(TokenType.RPAREN);

    expect(TokenType.VALUES);
    expect(TokenType.LPAREN);
    List<Object> values = parseValueList();
    expect(TokenType.RPAREN);

    return new
    InsertStatement(table, columns, values);
}
```

```
private List<Object> parseValueList() {
    List<Object> values = new ArrayList<>();
    values.add(parseLiteral());
    while (check(TokenType.COMMA)) {
        advance();
        values.add(parseLiteral());
    }
    return values;
}
```

```
private UpdateStatement parseUpdate() {
    expect(TokenType.UPDATE);
    String table = expect(IDENTIFIER).text();
    expect(TokenType.SET);

    Map<String, Object> assignments = new
    LinkedHashMap<>();
    assignments.putAll(parseAssignment());
    while (check(TokenType.COMMA)) {
        advance();
        assignments.putAll(parseAssignment());
    }
}
```

```
Expression where = check(WHERE) ?
parseWhereClause() : null;
return new UpdateStatement(table,
assignments, where);
}
```

```
private DeleteStatement parseDelete() {
    expect(TokenType.DELETE);
    expect(TokenType.FROM);
    String table = expect(IDENTIFIER).text();
    Expression where = check(WHERE) ?
    parseWhereClause() : null;
    return new DeleteStatement(table, where);
}
```

```

private CreateTableStatement
parseCreateTable() {
    expect(TokenType.CREATE);
    expect(TokenType.TABLE);
    String table =
expect(TokenType.IDENTIFIER).text();

    expect(TokenType.LPAREN);
    List<Column> columns = new ArrayList<>();
    columns.add(parseColumnDefinition());
    while (check(TokenType.COMMA)) {
        advance();
        columns.add(parseColumnDefinition());
    }
    expect(TokenType.RPAREN);

    return new CreateTableStatement(new
TableSchema(table, columns));
}

```

```

private DropTableStatement
parseDropTable() {
    expect(TokenType.DROP);
    expect(TokenType.TABLE);
    String table =
expect(TokenType.IDENTIFIER).text();
    return new DropTableStatement(table);
}

```

```

private Column parseColumnDefinition() {
    String name = expect(IDENTIFIER).text();
    DataType type =
DataType.valueOf(expect(IDENTIFIER).text().to
UpperCase());
    boolean isPrimaryKey = false;
    if (check(TokenType.PRIMARY)) {
        advance();
        expect(TokenType.KEY);
        isPrimaryKey = true;
    }
    return new Column(name, type,
!isPrimaryKey);
    // a primary key column is implicitly NOT
NULL
}

```

3.2. The WHERE Clause: A Small Expression AST

```
// ast/Expression.java
```

```
public sealed interface Expression permits Comparison, And, Or { }
```

```
public record Comparison(String column, String operator, Object value) implements Expression { }
```

```
public record And(Expression left, Expression right) implements Expression { }
```

```
public record Or(Expression left, Expression right) implements Expression { }
```

```
// SqlParser.java, continued – implements expression := comparison | expression  
AND expression | expression OR expression
```

```
private Expression parseWhereClause() {
```

```
    expect(TokenType.WHERE);
```

```
    return parseExpression();
```

```
}
```

```
private Expression parseExpression() {
```

```
    Expression left = parseComparison();
```

```
    while (check(TokenType.AND) || check(TokenType.OR)) {
```

```
        boolean isAnd = check(TokenType.AND);
```

```
        advance();
```

```
        Expression right = parseComparison();
```

```
        left = isAnd ? new And(left, right) : new Or(left, right);
```

```
    }
```

```
    return left;
```

```
}
```

```
private Comparison parseComparison() {
```

```
    String column = expect(TokenType.IDENTIFIER).text();
```

```
    String operator = parseOperator();
```

```
    Object value = parseLiteral();
```

```
    return new Comparison(column, operator, value);
```

```
}
```

```
private String parseOperator() {
```

```
    Token token = advance();
```

```
    return switch (token.type()) {
```

```
        case EQ -> "="; case NEQ -> "!="; case GT -> ">";
```

```
        case GTE -> ">="; case LT -> "<"; case LTE -> "<=";
```

```
        default -> throw new IllegalArgumentException("Expected a comparison operator but found " + token);
```

```
    };
```

```
}
```

salary > 50000 AND department = 'Engineering' parses into **And(Comparison("salary", ">", 50000), Comparison("department", "=", "Engineering"))** — a tree shape §13's evaluator walks recursively, matching the sealed interface's own shape exactly.

3.3. Implementing the Expression Evaluator

```
// ast/ExpressionEvaluator.java
public final class ExpressionEvaluator {

    public static boolean matches(Row row, Expression expression) {
        return switch (expression) {
            case Comparison c -> evaluateComparison(row.get(c.column()), c);
            case And a -> matches(row, a.left()) && matches(row, a.right());
            case Or o -> matches(row, o.left()) || matches(row, o.right());
        };
    }

    static boolean evaluateComparison(Object actual, Comparison c) {
        int cmp = compare(actual, c.value());
        return switch (c.operator()) {
            case "=" -> Objects.equals(actual, c.value());
            case "!=" -> !Objects.equals(actual, c.value());
            case ">" -> cmp > 0;
            case ">=" -> cmp >= 0;
            case "<" -> cmp < 0;
            case "<=" -> cmp <= 0;
            default -> throw new IllegalStateException("Unsupported operator: " + c.operator());
        };
    }

    @SuppressWarnings("unchecked")
    private static int compare(Object actual, Object expected) {
        if (actual instanceof Comparable<?> && expected != null) {
            return ((Comparable<Object>) actual).compareTo(expected);
        }
        return 0;
    }
}
```

Expression being a sealed interface means **matches's switch** needs no **default** branch — and the compiler refuses to build this file if a fourth expression type (say, a future **Not**) is ever added to the permits clause without a matching **case** here. That guarantee is what stops a new expression type from silently falling through to "matches nothing" instead of a compile error.

3.4. QueryResult and TransactionContext

Two small supporting types every executor and the dispatcher need:

```
public record QueryResult(List<Row> rows, int affectedRows) {

    public static QueryResult ofRows(List<Row> rows) {
        return new QueryResult(rows, rows.size());
    }

    public static QueryResult ofAffected(int count) {
        return new QueryResult(List.of(), count);
    }
}
```

```
public final class TransactionContext {
    // Deliberately empty for now. This guide focuses on parsing/planning/execution;
    // real isolation, locking, and rollback belong to a transaction manager, covered
    // in depth by the companion Storage Engine guide's crash-recovery and WAL sections.
    // Until a real TransactionContext is wired in, every statement is its own
    // implicit, auto-committed transaction.

    public static final TransactionContext AUTOCOMMIT = new TransactionContext();
}
```

Phase 4 — A Minimal StorageEngine to Execute Against

The executors need something real underneath them. This in-memory version is the Level 1 stand-in — the companion Storage Engine guide builds the real, page-based, durable implementation of this exact same interface.

```
public interface StorageEngine {
    void createTable(TableSchema schema);
    void dropTable(String tableName);
    void insert(String tableName, Row row);
    List<Row> scan(String tableName);
    void replaceRows(String tableName, List<Row> rows);
    Scan openScan(String tableName); // streaming access — used from Part 5 onward
}
```

```
public final class MemoryStorageEngine implements StorageEngine {
    private final Map<String, List<Row>> tables = new ConcurrentHashMap<>();
    @Override
    public void createTable(TableSchema schema) {
        tables.putIfAbsent(schema.tableName(), new CopyOnWriteArrayList<>());
    }

    @Override
    public void dropTable(String tableName) { tables.remove(tableName); }

    @Override
    public void insert(String tableName, Row row) { tables.get(tableName).add(row); }

    @Override
    public List<Row> scan(String tableName) { return List.copyOf(tables.get(tableName)); }

    @Override
    public void replaceRows(String tableName, List<Row> rows) {
        tables.put(tableName, new CopyOnWriteArrayList<>(rows));
    }

    @Override
    public Scan openScan(String tableName) {
        throw new UnsupportedOperationException("see Part 5");
    }
}
```

replaceRows is a deliberately blunt instrument: **UpdateExecutor/DeleteExecutor** (§17) compute a new row list and swap the whole thing in, rather than mutating individual rows — correct and simple here, and honestly wasteful once a table is large, which is exactly the problem the real page-based storage engine (the companion guide) solves properly.

The Six Statement Executors

```
public interface StatementExecutor<S extends SqlStatement> {  
    QueryResult execute(S statement, TransactionContext transaction);  
}
```

4.1. SelectExecutor

```
public final class SelectExecutor implements StatementExecutor<SelectStatement> {  
    private final StorageEngine storageEngine;  
  
    public SelectExecutor(StorageEngine storageEngine) {this.storageEngine = storageEngine; }  
  
    @Override  
    public QueryResult execute(SelectStatement statement, TransactionContext transaction) {  
        List<Row> matching = storageEngine.scan(statement.table()).stream()  
            .filter(row -> statement.where() == null  
                || ExpressionEvaluator.matches(row, statement.where()))  
            .toList();  
  
        if (statement.columns().size() == 1 && statement.columns().get(0).equals("")) {  
            return QueryResult.ofRows(matching);  
        }  
  
        List<Row> projected = matching.stream()  
            .map(row -> projectColumns(row, statement.columns()))  
            .toList();  
        return QueryResult.ofRows(projected);  
    }  
  
    private Row projectColumns(Row row, List<String> columns) {  
        Map<String, Object> projected = new LinkedHashMap<>();  
        for (String column : columns) {  
            projected.put(column, row.get(column));  
        }  
        return new Row(projected);  
    }  
}
```

4.2. InsertExecutor

```
public final class InsertExecutor implements StatementExecutor<InsertStatement> {

    private final StorageEngine storageEngine;
    private final Catalog catalog;

    public InsertExecutor(StorageEngine storageEngine, Catalog catalog) {
        this.storageEngine = storageEngine;
        this.catalog = catalog;
    }

    @Override
    public QueryResult execute(InsertStatement statement, TransactionContext transaction) {
        TableSchema schema = catalog.getTable(statement.table());
        if (schema == null) {
            throw new IllegalStateException("No such table: " + statement.table());
        }
        if (statement.columns().size() != statement.values().size()) {
            throw new IllegalArgumentException("Column count does not match value count");
        }

        Map<String, Object> values = new LinkedHashMap<>();
        for (int i = 0; i < statement.columns().size(); i++) {
            values.put(statement.columns().get(i), statement.values().get(i));
        }
        validateAgainstSchema(schema, values);

        storageEngine.insert(statement.table(), new Row(values));
        return QueryResult.ofAffected(1);
    }

    private void validateAgainstSchema(TableSchema schema, Map<String, Object> values) {
        for (Column column : schema.columns()) {
            if (!column.nullable() && values.get(column.name()) == null) {
                throw new IllegalStateException(column.name() + " cannot be null");
            }
        }
    }
}
```

4.3. UpdateExecutor

```
public final class UpdateExecutor implements StatementExecutor<UpdateStatement> {
    private final StorageEngine storageEngine;
    public UpdateExecutor(StorageEngine storageEngine) { this.storageEngine = storageEngine; }

    @Override
    public QueryResult execute(UpdateStatement statement, TransactionContext transaction) {
        List<Row> current = storageEngine.scan(statement.table());
        List<Row> updated = new ArrayList<>();
        int affected = 0;

        for (Row row : current) {
            if (statement.where() == null || ExpressionEvaluator.matches(row, statement.where())) {
                Map<String, Object> newValues = new LinkedHashMap<>();
                for (String column : row.columnNames()) {
                    newValues.put(column, row.get(column));
                }
                newValues.putAll(statement.assignments());
                updated.add(new Row(newValues));
                affected++;
            } else {
                updated.add(row);
            }
        }

        storageEngine.replaceRows(statement.table(), updated);
        return QueryResult.ofAffected(affected);
    }
}
```

4.4. DeleteExecutor:

```
public final class DeleteExecutor implements StatementExecutor<DeleteStatement> {
    private final StorageEngine storageEngine;
    public DeleteExecutor(StorageEngine storageEngine) {this.storageEngine = storageEngine; }

    @Override
    public QueryResult execute(DeleteStatement statement, TransactionContext transaction) {
        List<Row> current = storageEngine.scan(statement.table());
        List<Row> kept = new ArrayList<>();
        int affected = 0;

        for (Row row : current) {
            boolean shouldDelete = statement.where() == null
                || ExpressionEvaluator.matches(row, statement.where());
            if (shouldDelete) {
                affected++;
            } else {
                kept.add(row);
            }
        }
        storageEngine.replaceRows(statement.table(), kept);
        return QueryResult.ofAffected(affected);
    }
}
```

4.5. CreateTableExecutor:

```
public final class CreateTableExecutor implements StatementExecutor<CreateTableStatement> {

    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public CreateTableExecutor(Catalog catalog, StorageEngine storageEngine) {
        this.catalog = catalog;
        this.storageEngine = storageEngine;
    }

    @Override
    public QueryResult execute(CreateTableStatement statement, TransactionContext transaction) {
        if (catalog.tableExists(statement.schema().tableName())) {
            throw new IllegalStateException("Table already exists: " +
statement.schema().tableName());
        }
        catalog.createTable(statement.schema());
        storageEngine.createTable(statement.schema());
        return QueryResult.ofAffected(0);
    }
}
```

4.6. DropTableExecutor:

```
public final class DropTableExecutor implements StatementExecutor<DropTableStatement> {

    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public DropTableExecutor(Catalog catalog, StorageEngine storageEngine) {
        this.catalog = catalog;
        this.storageEngine = storageEngine;
    }

    @Override
    public QueryResult execute(DropTableStatement statement, TransactionContext transaction) {
        if (!catalog.tableExists(statement.table())) {
            throw new IllegalStateException("No such table: " + statement.table());
        }
        catalog.dropTable(statement.table());
        storageEngine.dropTable(statement.table());
        return QueryResult.ofAffected(0);
    }
}
```

4.7. The QueryEngine Dispatcher: Wiring Lexer, Parser, and Executors Together

This is the answer to "how do lexer, parser, and executors actually connect":

```
public final class QueryEngine {

    private final SelectExecutor selectExecutor;
    private final InsertExecutor insertExecutor;
    private final UpdateExecutor updateExecutor;
    private final DeleteExecutor deleteExecutor;
    private final CreateTableExecutor createTableExecutor;
    private final DropTableExecutor dropTableExecutor;

    public QueryEngine(Catalog catalog, StorageEngine storageEngine) {
        this.selectExecutor = new SelectExecutor(storageEngine);
        this.insertExecutor = new InsertExecutor(storageEngine, catalog);
        this.updateExecutor = new UpdateExecutor(storageEngine);
        this.deleteExecutor = new DeleteExecutor(storageEngine);
        this.createTableExecutor = new CreateTableExecutor(catalog, storageEngine);
        this.dropTableExecutor = new DropTableExecutor(catalog, storageEngine);
    }

    public QueryResult run(String sql) {
        List<Token> tokens = new SqlLexer(sql).tokenize();           // §7
        SqlStatement statement = new SqlParser(tokens).parseStatement(); // §11
        return execute(statement, TransactionContext.AUTOCOMMIT);
    }

    public QueryResult execute(SqlStatement statement, TransactionContext transaction) {
        return switch (statement) {
            case SelectStatement s -> selectExecutor.execute(s, transaction);
            case InsertStatement s -> insertExecutor.execute(s, transaction);
            case UpdateStatement s -> updateExecutor.execute(s, transaction);
            case DeleteStatement s -> deleteExecutor.execute(s, transaction);
            case CreateTableStatement s -> createTableExecutor.execute(s, transaction);
            case DropTableStatement s -> dropTableExecutor.execute(s, transaction);
        };
    }
}
```

Alternative: ANTLR-Generated Parser + the Visitor Pattern

```
// Uses an ANTLR-generated MySQLLexer/MySQLStatementParser instead of a
// hand-written one. ANTLR turns a .g4 grammar file into the lexer,
public class MySqlParser implements IParser {
    MySqlStatementVisitor sqlStatementVisitor;

    public MySqlParser(String sql) {
        MySQLLexer lexer = new MySQLLexer(CharStreams.fromString(sql));
        var parser = new MySQLStatementParser(new CommonTokenStream(lexer));

        sqlStatementVisitor = new MySqlStatementVisitor(parser);
        sqlStatementVisitor.visit(parser.execute());
    }

    @Override
    public QueryData queryCmd() {return (QueryData) sqlStatementVisitor.getValue();}

    @Override
    public Object updateCmd() {return sqlStatementVisitor.getValue(); }
}

// The Visitor walks ANTLR's generated parse tree and extracts exactly
// the fields needed into a domain object -- the ANTLR equivalent of
// this guide's SqlStatement records.
public class MySqlStatementVisitor extends MySQLStatementBaseVisitor {
    private final MySQLStatementParser parser;
    private COMMAND_TYPE commandType;
    private String tableName;

    @Override
    public Object visitCreateTable(MySQLStatementParser.CreateTableContext ctx) {
        commandType = COMMAND_TYPE.CREATE_TABLE;
        return super.visitCreateTable(ctx);
    }

    @Override
    public Object visitTableName(MySQLStatementParser.TableNameContext ctx) {
        this.tableName = ctx.name().getText();
        return super.visitTableName(ctx);
    }
}
```

The pipeline is structurally identical to section 10.1's, just built from generated pieces instead of hand-written ones:

This guide (hand-written):	ANTLR-based (e.g. ShardingSphere):
SQL text	SQL text
SqlLexer (hand-written)	MySQLLexer (generated from a grammar)
SqlParser (hand-written)	MySQLStatementParser (generated) + a
	Visitor subclass that walks its parse
tree	
SqlStatement (a sealed	QueryData / InsertData / ModifyData
interface of records)	(plain domain objects the Visitor builds)
QueryEngine.execute(...)	dispatch by commandType / object type
(an exhaustive switch)	(structurally the same idea, without
	a sealed-type-checked switch

5. Why Direct Execution Doesn't Scale: Introducing the Planner

§17's executors work correctly, and they share one real limitation: **SelectExecutor.execute** always calls **storageEngine.scan(table)** — a full table scan, every single time, even for **WHERE id = 100** against a table with a primary-key index that could answer the same question in **$O(\log n)$** instead of **$O(n)$** . Nothing in §17's code has anywhere to record "use the index here" as a decision — the scan-then-filter logic is hard-coded inline. Fixing this requires separating "what to do" (a **Plan**, an object you can inspect, compare, and choose between) from "just doing it" — which is the entire reason a query planner exists.

The Three Parts of a Query Engine

A query engine is not one component. It is three, each with a distinct job:

Query Optimizer	-- decides HOW to answer a query (which plan, which indexes)
Execution Engine	-- actually reads rows/columns according to that plan
Catalog / Metadata	-- knows what tables and indexes exist, and their shape Manager

Part 4 (this part) builds the Optimizer's mechanism and the seam it plugs into. Part 5 builds the Execution Engine. Part 6 builds the Catalog Manager for real. Keeping these three separate is the same Interface Segregation discipline (a **QueryPlanner**, a **QueryExecutor**, and a **Catalog** as three different interfaces) that motivates never merging them into one **DatabaseEverything** class.

Rule-Based vs Cost-Based Optimization

"Use the index if the **WHERE** clause matches an indexed column, otherwise scan the table" is **Rule-Based Optimization (RBO)** — a fixed rule, applied without comparing alternatives. A **Cost-Based Optimizer (CBO)** generalizes this: instead of one hard-coded rule, it generates several candidate plans and picks whichever has the lowest estimated cost, using statistics the Catalog Manager (Part 6) collects — row counts, blocks accessed, selectivity. RBO is simpler to build and reason about; CBO scales better as the number of indexes and join orderings grows, at the cost of needing real statistics to estimate against. This guide builds RBO's actual mechanism — §27's **Plan.blocksAccessed()/recordsOutput()** are exactly the hooks a later CBO would compare across candidate plans.

5.1. The Plan Interface

```
public interface Plan {
    Scan open();           // §29 -- calling this does not read any data yet, only prepares to
    int blocksAccessed(); // cost estimate, for a future CBO to compare across candidate plans
    int recordsOutput();
    TableSchema schema();
}
```

5.2. **TablePlan** — the only **Plan** that talks to **StorageEngine** directly:

```
public final class TablePlan implements Plan {
    private final String tableName;
    private final StorageEngine storageEngine;
    private final Catalog catalog;

    public TablePlan(String tableName, StorageEngine storageEngine, Catalog catalog) {
        this.tableName = tableName;
        this.storageEngine = storageEngine;
        this.catalog = catalog;
    }

    @Override
    public Scan open() { return storageEngine.openScan(tableName); }

    @Override
    public int blocksAccessed() { return catalog.getStatistics(tableName).blockCount(); }

    @Override
    public int recordsOutput() { return catalog.getStatistics(tableName).recordCount(); }

    @Override
    public TableSchema schema() { return catalog.getTable(tableName); }
}
```

5.3. **SelectPlan** and **ProjectPlan** — each wraps another Plan, the way a Decorator wraps another object, rather than talking to storage itself:

```
public final class SelectPlan implements Plan {
    private final Plan input;
    private final Expression predicate;

    public SelectPlan(Plan input, Expression predicate) {
        this.input = input;
        this.predicate = predicate;
    }

    @Override
    public Scan open() { return new SelectScan(input.open(), predicate); } // §30

    @Override // // filtering reads the SAME blocks -- no extra I/O
    public int blocksAccessed() { return input.blocksAccessed(); }

    @Override // // a naive fixed selectivity -- a CBO refines this
    public int recordsOutput() { return input.recordsOutput() / 2; }

    @Override // // filtering ROWS never changes which COLUMNS exist
    public TableSchema schema() { return input.schema(); }
}
```

5.4. ProjectPlan

```
public final class ProjectPlan implements Plan {
    private final Plan input;
    private final List<String> fields;

    public ProjectPlan(Plan input, List<String> fields) {
        this.input = input;
        this.fields = fields;
    }

    @Override // // §30
    public Scan open() { return new ProjectScan(input.open(), fields); }

    @Override
    public int blocksAccessed() { return input.blocksAccessed(); }

    @Override // // projecting columns doesn't change row COUNT
    public int recordsOutput() { return input.recordsOutput(); }

    @Override // // a NARROWER schema -- fewer columns
    public TableSchema schema() { return input.schema().project(fields); }
}
```

TablePlan, **SelectPlan**, and **ProjectPlan** compose exactly the way **SelectStatement's** three parts (table, where, columns) already suggest: read, then filter, then narrow columns — each step wrapping the previous **Plan**, never reaching around it to touch storage directly.

5.5. Phase 10 — BasicQueryPlanner and UpdatePlanner

```
public interface QueryPlanner {
    Plan createPlan(SelectStatement statement, TransactionContext tx);
}
```

```
public final class BasicQueryPlanner implements QueryPlanner {
    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public BasicQueryPlanner(Catalog catalog, StorageEngine storageEngine) {
        this.catalog = catalog;
        this.storageEngine = storageEngine;
    }

    @Override
    public Plan createPlan(SelectStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog); // Step 1: read

        if (statement.where() != null) plan = new SelectPlan(plan, statement.where()); // Step 2: filter
        rows

        boolean selectStar = statement.columns().size() == 1 &&
        statement.columns().get(0).equals("");
        if (!selectStar) plan = new ProjectPlan(plan, statement.columns()); // Step 3: filter columns

        return plan;
    }
}
```

Compare this three-step body to §17's **SelectExecutor** — the logic (read, then filter, then project) is identical. What changed is that each step is now a composed, reusable **Plan** object instead of an inline **.stream().filter(...)** call — the exact shape a Cost-Based Optimizer needs later, since it can only compare plans it can hold as objects, not inline expressions buried in an executor's method body.

```

public interface UpdatePlanner {
    int executeInsert(InsertStatement statement, TransactionContext tx);
    int executeUpdate(UpdateStatement statement, TransactionContext tx);
    int executeDelete(DeleteStatement statement, TransactionContext tx);
    int executeCreateTable(CreateTableStatement statement, TransactionContext tx);
    int executeDropTable(DropTableStatement statement, TransactionContext tx);
}

```

```

public final class BasicUpdatePlanner implements UpdatePlanner {
    private final Catalog catalog; // constructor injection
    private final StorageEngine storageEngine; // constructor injection

    public int executeInsert(InsertStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
        Scan scan = plan.open();
        scan.insert(); // position at a fresh slot
        for (int i = 0; i < statement.columns().size(); i++) {
            scan.setVal(statement.columns().get(i), statement.values().get(i));
        }
        scan.close();
        return 1;
    }

    public int executeUpdate(UpdateStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
        if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
        Scan scan = plan.open();
        int affected = 0;
        while (scan.next()) {
            for (var entry : statement.assignments().entrySet())
                scan.setVal(entry.getKey(), entry.getValue());
            affected++;
        }
        scan.close();
        return affected;
    }

    public int executeDelete(DeleteStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
        if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
        Scan scan = plan.open();
        int affected = 0;
        while (scan.next()) { scan.delete(); affected++; }
        scan.close();
        return affected;
    }

    public int executeCreateTable(CreateTableStatement statement, TransactionContext tx) {
        catalog.createTable(statement.schema());
        storageEngine.createTable(statement.schema());
        return 0;
    }

    public int executeDropTable(DropTableStatement statement, TransactionContext tx) {
        catalog.dropTable(statement.table());
        storageEngine.dropTable(statement.table());
        return 0;
    }
}

```

5.7. Phase 11 — BasicPlanner: A Facade Over Parsing and Planning

BasicPlanner is a **Facade** sitting between the parser and the two planners above — the same seam idea as §18's **QueryEngine**, one layer deeper now that statements produce **Plans** instead of being executed immediately:

```
public final class BasicPlanner {

    private final QueryPlanner queryPlanner;
    private final UpdatePlanner updatePlanner;

    public BasicPlanner(QueryPlanner queryPlanner, UpdatePlanner updatePlanner) {
        this.queryPlanner = queryPlanner;
        this.updatePlanner = updatePlanner;
    }

    public Plan createQueryPlan(String sql, TransactionContext tx) {
        SqlStatement statement = parse(sql);
        if (!(statement instanceof SelectStatement select))
            throw new IllegalArgumentException("Not a query: " + sql);
        return queryPlanner.createPlan(select, tx);
    }

    public int executeUpdate(String sql, TransactionContext tx) {
        SqlStatement statement = parse(sql);
        return switch (statement) {
            case InsertStatement s -> updatePlanner.executeInsert(s, tx);
            case UpdateStatement s -> updatePlanner.executeUpdate(s, tx);
            case DeleteStatement s -> updatePlanner.executeDelete(s, tx);
            case CreateTableStatement s -> updatePlanner.executeCreateTable(s, tx);
            case DropTableStatement s -> updatePlanner.executeDropTable(s, tx);
            case SelectStatement s -> throw new IllegalArgumentException("Use createQueryPlan for a
SELECT");
        };
    }

    private SqlStatement parse(String sql) {
        List<Token> tokens = new SqlLexer(sql).tokenize(); // §7
        return new SqlParser(tokens).parseStatement();    // §11
    }
}
```

6. Phase 12 — BasicQueryEngine: doQuery and doUpdate

The externally-visible database facade — what a CLI or a JDBC driver actually calls:

```
public final class BasicQueryEngine {  
    private final BasicPlanner planner;  
    private final TransactionManager transactionManager; // see the companion Storage Engine  
    guide  
  
    public BasicQueryEngine(BasicPlanner planner, TransactionManager transactionManager) {  
        this.planner = planner;  
        this.transactionManager = transactionManager;  
    }  
  
    public QueryResult doQuery(String sql) {  
        TransactionContext tx = transactionManager.begin();  
        Plan plan = planner.createQueryPlan(sql, tx);  
        Scan scan = plan.open();  
  
        List<Row> rows = new ArrayList<>();  
        while (scan.next()) {  
            Map<String, Object> values = new LinkedHashMap<>();  
            for (String field : plan.schema().fieldNames()) values.put(field, scan.getVal(field));  
            rows.add(new Row(values));  
        }  
        scan.close();  
        transactionManager.commit(tx);  
        return QueryResult.ofRows(rows);  
    }  
  
    public QueryResult doUpdate(String sql) {  
        TransactionContext tx = transactionManager.begin();  
        int affected = planner.executeUpdate(sql, tx);  
        transactionManager.commit(tx);  
        return QueryResult.ofAffected(affected);  
    }  
}
```

doQuery is the only place left that walks a **Scan** into a **List<Row>** — and it does that only because the caller above it wants a materialized result set to print or return. The engine itself, all the way down through **Plan/Scan**, never materializes more than one row at a time (§32 explains exactly why that matters).

Revisiting the Six Executors to Use the Planner

```
// SelectExecutor, REVISITED: same StatementExecutor<SelectStatement> contract as §17,  
// but the BODY now delegates to the planner instead of scanning+filtering a  
// materialized List<Row> by hand.  
public final class SelectExecutor implements StatementExecutor<SelectStatement> {  
  
    private final QueryPlanner queryPlanner;  
  
    public SelectExecutor(QueryPlanner queryPlanner) { this.queryPlanner = queryPlanner; }  
  
    @Override  
    public QueryResult execute(SelectStatement statement, TransactionContext transaction) {  
        Plan plan = queryPlanner.createPlan(statement, transaction);  
        Scan scan = plan.open();  
  
        List<Row> rows = new ArrayList<>();  
        while (scan.next()) {  
            Map<String, Object> values = new LinkedHashMap<>();  
            for (String field : plan.schema().fieldNames()) values.put(field, scan.getVal(field));  
            rows.add(new Row(values));  
        }  
        scan.close();  
        return QueryResult.ofRows(rows);  
    }  
}
```

The public contract did not change at all. Only what happens inside **execute** changed — from "load everything, then filter in Java" to "compose a plan, then stream through it." This is precisely why §17 was told to depend on **Catalog/StorageEngine** interfaces: the same Dependency Inversion that let **MemoryStorageEngine** be swapped for a real one also lets an executor's implementation strategy change without its callers noticing.

Phase 13 — The Scan Interface: Streaming Instead of Materializing

`StorageEngine.scan(tableName)` returns a `List<Row>` — every row, loaded into memory, before anything even looks at the first one. That is fine for a table that fits comfortably in memory; it is a real problem the moment a table doesn't. A `Scan` fixes this by being an iterator, not a collection — it produces rows one at a time, on demand:

```
public interface Scan {  
    boolean next();  
    Object getVal(String fieldName);  
    boolean hasField(String fieldName);  
    void close();  
  
    // Writable operations -- a stricter design (see the note below) splits these  
    // into a separate interface so a read-only SELECT path can never call them.  
    void insert();  
    void setVal(String fieldName, Object value);  
    void delete();  
}
```

Putting both read and write methods on one interface is a pragmatic simplification, not the final answer — Interface Segregation argues against exactly this shape in the abstract. A stricter design splits this into **ReadOnlyScan** (`next/getVal/hasField/close`) and **UpdatableScan** extends **ReadOnlyScan** (adding `insert/setVal/delete`), so a **SELECT** path is never even able to call `delete()` by accident — the compiler enforces it instead of a runtime check. §29's implementations work identically either way; only which interface each one is declared to return changes.

Phase 14 — TableScan

TableScan is the only Scan that talks to **StorageEngine** directly — everything else wraps another Scan:

```
// StorageEngine (§16) gains one method beyond insert()/scan()/replaceRows():
public interface StorageEngine {
    // ... createTable, dropTable, insert, scan, replaceRows from §16 ...
    Scan openScan(String tableName);
}
```

```
// MemoryStorageEngine.openScan -- still backed by the in-memory List<Row> for now.
// Once real page-based storage exists (the companion Storage Engine guide), this walks
// Pages via RowId instead of a Java Iterator -- the Scan CONTRACT above does not change.
@Override
public Scan openScan(String tableName) {
    List<Row> rows = tables.get(tableName);
    return new Scan() {
        private int index = -1;

        @Override
        public boolean next() {
            index++;
            return index < rows.size();
        }

        @Override
        public Object getVal(String fieldName) { return rows.get(index).get(fieldName); }

        @Override
        public boolean hasField(String fieldName) {
            return rows.get(index).columnNames().contains(fieldName);
        }

        @Override
        public void insert() {
            rows.add(new Row(new HashMap<>()));
            index = rows.size() - 1;
        }

        @Override
        public void setVal(String fieldName, Object value) {
            Map<String, Object> updated = new LinkedHashMap<>();
            for (String column : rows.get(index).columnNames())
                updated.put(column, rows.get(index).get(column));
            updated.put(fieldName, value);
            rows.set(index, new Row(updated));
        }

        @Override
        public void delete() {
            rows.remove(index);
            index--; // the next next() call must not skip the row that just slid into this slot
        }

        @Override
        public void close() {}
    };
}
```

Phase 15 — SelectScan and ProjectScan

SelectScan wraps a child **Scan** and a predicate — it filters rows as they stream past, one at a time, never holding more than the current row:

```
public final class SelectScan implements Scan {
    private final Scan input;
    private final Expression predicate;

    public SelectScan(Scan input, Expression predicate) {
        this.input = input;
        this.predicate = predicate;
    }

    @Override
    public boolean next() {
        while (input.next()) {
            if (ExpressionEvaluator.matchesScan(this, predicate)) return true; // §31
        }
        return false;
    }

    @Override public Object getVal(String fieldName) { return input.getVal(fieldName); }
    @Override public boolean hasField(String fieldName) { return input.hasField(fieldName); }
    @Override public void setVal(String fieldName, Object value) { input.setVal(fieldName, value); }
    @Override public void insert() { input.insert(); }
    @Override public void delete() { input.delete(); }
    @Override public void close() { input.close(); }
}
```

```
public final class ProjectScan implements Scan {
    private final Scan input;
    private final List<String> fields;

    @Override
    public boolean next() { return input.next(); }

    @Override
    public Object getVal(String fieldName) {
        if (!hasField(fieldName)) throw new RuntimeException("field " + fieldName + " not found.");
        return input.getVal(fieldName);
    }

    @Override
    public boolean hasField(String fieldName) { return fields.contains(fieldName); }

    @Override public void setVal(String fieldName, Object value) { throw new
    UnsupportedOperationException("read-only projection"); }
    @Override public void insert() { throw new UnsupportedOperationException("read-only
    projection"); }
    @Override public void delete() { throw new UnsupportedOperationException("read-only
    projection"); }
    @Override public void close() { input.close(); }
}
```

ProjectScan throwing on every write method is worth noticing — projection is fundamentally a read-shaping operation, and §28's "split into **ReadOnlyScan/UpdatableScan**" refinement would let this class simply not implement the write methods at all, rather than implementing them just to reject every call.

31. Extending the Expression Evaluator to Work Against a Scan

SelectScan (§30) needs to evaluate the same **Expression** tree §14 already built — but against a **Scan**'s current position, not a fully-formed **Row**. Extend **ExpressionEvaluator** with a second entry point that mirrors **matches(Row, Expression)** field for field, reading through **Scan.getVal(...)** instead of **Row.get(...)**:

```
// Added to ExpressionEvaluator (§14):
public static boolean matchesScan(Scan scan, Expression expression) {
    return switch (expression) {
        case Comparison c -> evaluateComparison(scan.getVal(c.column()), c);
        case And a -> matchesScan(scan, a.left()) && matchesScan(scan, a.right());
        case Or o -> matchesScan(scan, o.left()) || matchesScan(scan, o.right());
    };
}
```

§14's **evaluateComparison** was already written to take the actual value directly (not a **Row**), specifically so both **matches(Row, ...)** and **matchesScan(Scan, ...)** can share that one comparison method instead of duplicating it — one comparison implementation, two entry points, which is what keeps **Row**-based execution (Part 3) and **Scan**-based execution (Part 5) from silently drifting into two different definitions of "matches."

32. Why Streaming Beats Materializing: A Concrete Cost Trace

Trace what happens for: "SELECT name FROM employee WHERE salary > 50000;" against a table of one million rows where ten thousand match:

```
Part 3's original SelectExecutor (§17):
storageEngine.scan("employee") → loads ALL 1,000,000 rows into a List<Row>
    .stream().filter(...)        → then filters down to 10,000, still in memory
    .map(...)                    → then projects, producing a THIRD list

Part 4-5's Plan/Scan pipeline (§27-30):
TableScan.next() -> SelectScan.next() → ProjectScan.next()
-- pulls ONE row at a time through all three stages, filters it, projects it, and only
the caller (BasicQueryEngine.doQuery, §26) decides whether to keep it in a List --
the pipeline itself never holds more than one row per stage at once.
```

Both eventually produce the same 10,000-row result. The difference is **peak memory**: one materializes three full intermediate collections along the way; the other never holds more than a handful of rows at any given instant, regardless of whether the table has a thousand rows or a billion. This is the concrete, measurable payoff of the Iterator-shaped **Scan** interface (§28) over §17's original "load a list, then **.stream()** it" approach — and it is also exactly why **Sort** and **Aggregate** (Part 9's future work) are structurally different from **Filter/Project**: sorting the whole result requires seeing every row before producing the first one, so a **SortScan** cannot stream the way **SelectScan/ProjectScan** do — a real, unavoidable limit on how far the streaming idea extends, worth stating honestly rather than implying every operator can be lazy for free.

33. Phase 16 — The Catalog Interface, Revisited

§9's **Catalog** interface has been used throughout this guide without a real, durable implementation — every example so far implicitly assumes something like a **Map<String, TableSchema>** in memory, which forgets every table the instant the process restarts. Part 6 fixes that properly.

```
public interface Catalog {
    void createTable(TableSchema schema);
    void dropTable(String tableName);
    TableSchema getTable(String tableName);
    boolean tableExists(String tableName);
    List<TableSchema> listTables();
    TableStatistics getStatistics(String tableName); // consumed by
    TablePlan.blocksAccessed()/recordsOutput(), §23
}

public record TableStatistics(int blockCount, int recordCount) { }
```

34. Phase 17 — TablePhysicalLayout: From Logical Schema to Byte Offsets

TableSchema (§9) is a table's logical shape — names, types, nullability. Computing a row's exact physical layout needs one more thing: the byte offset of each column within a row, and the row's total **slot size**:

```
public final class TablePhysicalLayout {
    private final Map<String, Integer> offsets = new HashMap<>();
    private final int slotSize;

    public TablePhysicalLayout(TableSchema schema) {
        int position = 0;
        for (Column column : schema.columns()) {
            offsets.put(column.name(), position);
            position += sizeInBytes(column);
        }
        this.slotSize = position;
    }

    private int sizeInBytes(Column column) {
        return switch (column.type()) {
            case INT -> 4;
            case BIGINT -> 8;
            case BOOLEAN -> 1;
            case VARCHAR -> 50; // fixed-length slots for simplicity -- even VARCHAR gets a fixed
budget
            case DECIMAL -> 12;
            case DATE, TIMESTAMP -> 8;
        };
    }

    public int offset(String fieldName) { return offsets.get(fieldName); }
    public int slotSize() { return slotSize; }
}
```

Fixed-length slots (even for **VARCHAR**) is a deliberate simplifying decision — worth restating because it's precisely what makes **offset(fieldName)** a single, cheap arithmetic lookup instead of "scan every earlier column to find where this one starts." The companion **Storage Engine guide** uses exactly this layout to encode/decode rows at the byte level.

35. Phase 18 — PersistentCatalog: Storing Schema as Data

A real database stores its own schema as ordinary rows, in special tables the database itself manages, so table definitions get the exact same durability (WAL, covered in the companion Storage Engine guide) as user data, instead of a separate, ad hoc persistence mechanism:

tinydb_tables			tinydb_columns			
tblname	slotsize		tblname	fldname	type	length
employee	48		employee	id	BIGINT	0
			employee	name	VARCHAR	50

```
public final class PersistentCatalog implements Catalog {
    private static final TableSchema TABLES_SCHEMA = new TableSchema("tinydb_tables", List.of(
        new Column("tblname", DataType.VARCHAR, false), new Column("slotsize", DataType.INT, false))
    );

    private static final TableSchema COLUMNS_SCHEMA = new TableSchema("tinydb_columns", List.of(
        new Column("tblname", DataType.VARCHAR, false), new Column("fldname", DataType.VARCHAR,
false),
        new Column("type", DataType.VARCHAR, false), new Column("length", DataType.INT, false))
    );

    private final StorageEngine storageEngine;

    public PersistentCatalog(StorageEngine storageEngine) {
        this.storageEngine = storageEngine;
        storageEngine.createTable(TABLES_SCHEMA); // bootstrap: the catalog's own tables must exist first
        storageEngine.createTable(COLUMNS_SCHEMA);
    }

    @Override
    public void createTable(TableSchema schema) {
        int slotSize = new TablePhysicalLayout(schema).slotSize(); // §34
        storageEngine.insert(TABLES_SCHEMA.tableName(),
            new Row(Map.of("tblname", schema.tableName(), "slotsize", slotSize)));
        for (Column column : schema.columns()) {
            storageEngine.insert(COLUMNS_SCHEMA.tableName(), new Row(Map.of(
                "tblname", schema.tableName(), "fldname", column.name(),
                "type", column.type().name(), "length", 0)));
        }
    }

    @Override
    public TableSchema getTable(String tableName) {
        boolean exists = storageEngine.scan(TABLES_SCHEMA.tableName()).stream()
            .anyMatch(row -> row.get("tblname").equals(tableName));
        if (!exists) return null;

        List<Column> columns = storageEngine.scan(COLUMNS_SCHEMA.tableName()).stream()
            .filter(row -> row.get("tblname").equals(tableName))
            .map(row -> new Column((String) row.get("fldname"), DataType.valueOf((String) row.get("type")),
true))
            .toList();
        return new TableSchema(tableName, columns);
    }

    // dropTable/tableExists/listTables/getStatistics omitted for brevity -- same
    // "read from tinydb_tables / tinydb_columns" pattern as getTable() above.
}
```

36. Follow-up: How Does the Planner Actually Choose an IndexScan?

§22 named Rule-Based Optimization's canonical example — "use the index if the **WHERE** clause matches an indexed column" — without yet showing the mechanism. With **Plan** (§23) as a real abstraction, the answer is now concrete: **BasicQueryPlanner.createPlan** (§24) needs one more decision point, made before defaulting to **TablePlan**:

```
@Override
public Plan createPlan(SelectStatement statement, TransactionContext tx) {
    Plan plan = selectAccessPath(statement); // NEW: choose TablePlan OR IndexSelectPlan, per §38

    if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
    boolean selectStar = statement.columns().size() == 1 && statement.columns().get(0).equals("");
    if (!selectStar) plan = new ProjectPlan(plan, statement.columns());
    return plan;
}

private Plan selectAccessPath(SelectStatement statement) {
    if (statement.where() instanceof Comparison c
        && c.operator().equals("=")
        && catalog.hasIndex(statement.table(), c.column())) {
        return new IndexSelectPlan(statement.table(), c.column(), c.value(), catalog); // §38
    }
    return new TablePlan(statement.table(), storageEngine, catalog); // the RBO "default" rule
}
```

catalog.hasIndex(table, column) is the one new **Catalog** method this decision needs — a Cost-Based Optimizer would replace this single **if** with a comparison of **blocksAccessed()** across every viable **Plan**, but the RBO version above is a real, working decision, not a placeholder.

37. Sketch: An IndexSelectPlan

```
public final class IndexSelectPlan implements Plan {
    private final String tableName;
    private final String indexedColumn;
    private final Object searchKey;
    private final Catalog catalog;

    @Override
    public Scan open() {
        RowId rowId = catalog.lookupIndex(tableName, indexedColumn, searchKey); // O(log n), not O(n)
        return new SingleRowScan(tableName, rowId); // a Scan over exactly one row, or zero if not found
    }

    @Override // a B+Tree lookup touches a small, roughly-constant number of pages
    public int blocksAccessed() { return 3; }

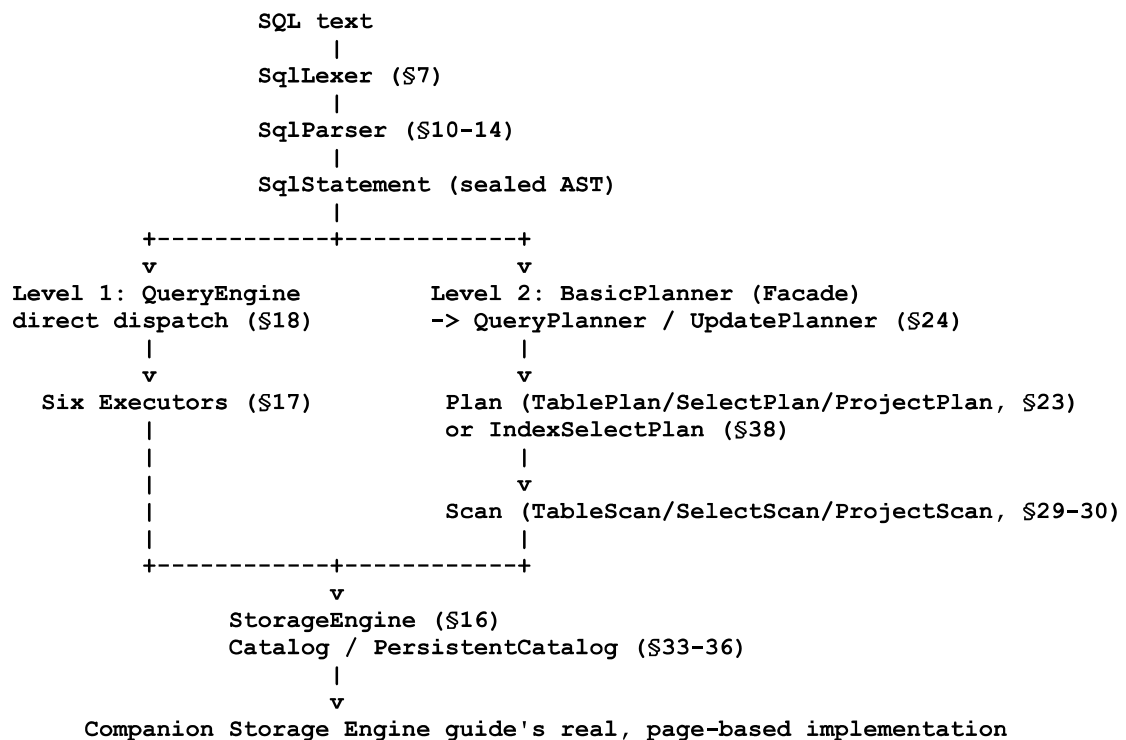
    @Override // an equality lookup on a unique index returns at most one row
    public int recordsOutput() { return 1; }

    @Override
    public TableSchema schema() { return catalog.getTable(tableName); }
}
```

Full Worked Example

```
"SELECT name FROM employee WHERE salary > 50000"
-> SqlLexer.tokenize()                                ($7)
-> SqlParser.parseStatement() -> SelectStatement      ($10-11, $13)
-> BasicQueryEngine.doQuery(sql)                      ($26)
    -> BasicPlanner.createQueryPlan(sql, tx)          ($25)
        -> BasicQueryPlanner.createPlan(statement, tx) ($24, extended by
$37)
            -> selectAccessPath -> TablePlan (no index on "salary")
            -> wrapped in SelectPlan(where: salary > 50000)
            -> wrapped in ProjectPlan(columns: ["name"])
        -> plan.open() -> ProjectScan(SelectScan(TableScan)) ($29-30)
        -> while (scan.next()) { ... }                ($26)
-> QueryResult.ofRows([{name: "Alice"}])
```

Final Architecture



Design Patterns Used

Pattern	Applied to
Command	Each <code>SqlStatement</code> is a self-contained, immutable description of an action
Facade	<code>BasicPlanner/BasicQueryEngine</code> hiding the parser, planner, and executor layers behind <code>doQuery/doUpdate</code>
Decorator	<code>SelectPlan/ProjectPlan</code> and <code>SelectScan/ProjectScan</code> each wrap another <code>Plan/Scan</code>
Iterator	<code>Scan</code> streams rows one at a time, exactly like <code>java.util.Iterator</code>
Strategy	Choosing <code>TablePlan</code> vs <code>IndexSelectPlan</code> per query, without the caller knowing which was chosen
Visitor	ANTLR's generated parse-tree traversal

SOLID Principles Applied

Principle	Where it holds
Single Responsibility	<code>SqlLexer</code> only tokenizes; <code>SqlParser</code> only parses; each executor only executes one statement type
Open/Closed	Adding a seventh <code>SqlStatement</code> type is additive everywhere the sealed interface is switched over — the compiler enforces it (§18, §25)
Liskov Substitution	Any <code>Plan/Scan</code> implementation is fully interchangeable from its caller's point of view
Interface Segregation	<code>QueryPlanner/UpdatePlanner/Catalog</code> are separate, narrow interfaces; §28 names the further split <code>Scan</code> itself deserves
Dependency Inversion	Every executor and planner depends on <code>Catalog/StorageEngine</code> interfaces, never a concrete implementation

Suggested Future Enhancements

Enhancement	Where it plugs in
JOIN support	A <code>JoinPlan/NestedLoopJoinScan</code> pair, composed the same way <code>SelectPlan/SelectScan</code> are
ORDER BY	A <code>SortPlan/SortScan</code> — the one operator §32 already flags as unable to stream
GROUP BY / aggregate functions	An <code>AggregatePlan</code> consuming a full (possibly sorted) input before producing output rows
Real B+Tree-backed indexes	See the companion <code>Storage Engine</code> guide's indexing sections
A real Cost-Based Optimizer	Extending §37's <code>selectAccessPath</code> to build several plans and pick the cheapest via <code>blocksAccessed()</code>
Prepared statements	Parameterizing <code>Comparison.value()</code> instead of baking a literal into the AST at parse time